Lifelike Motion

IKFootPlacement - Documentation

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

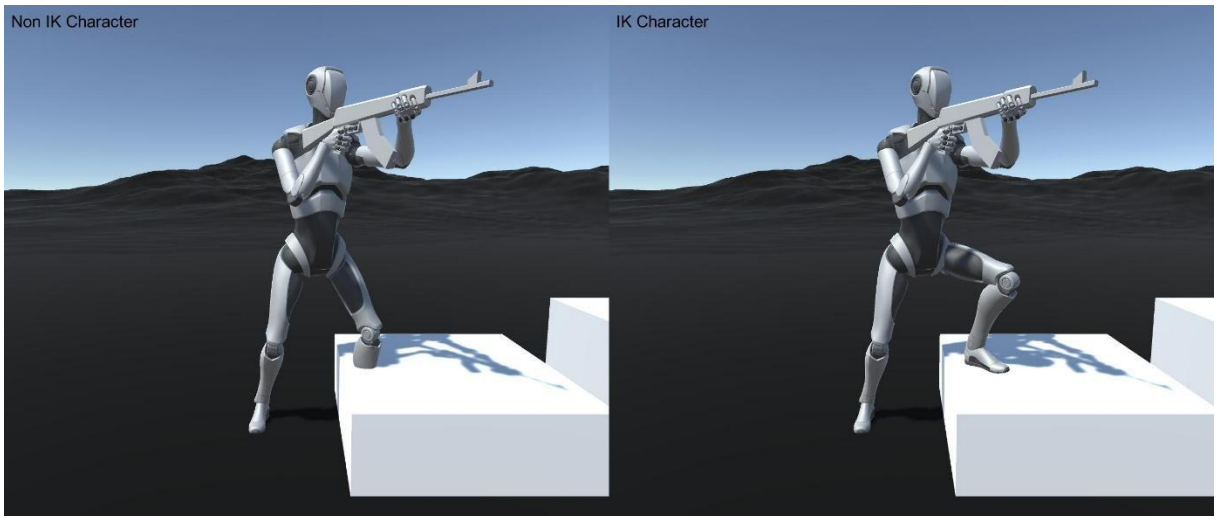
Table of Contents

1. Overview
2. Package Contents
3. Package Requirements
    3.1. Package requirements and dependencies
    3.2. Optional packages
4. Installation and Configuration
    4.1. Package import
    4.2. Script configuration
5. Usage
    5.1. Baking the animation onto the IK controls
    5.2. Using the IKFootPlacement script
6. Troubleshooting

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

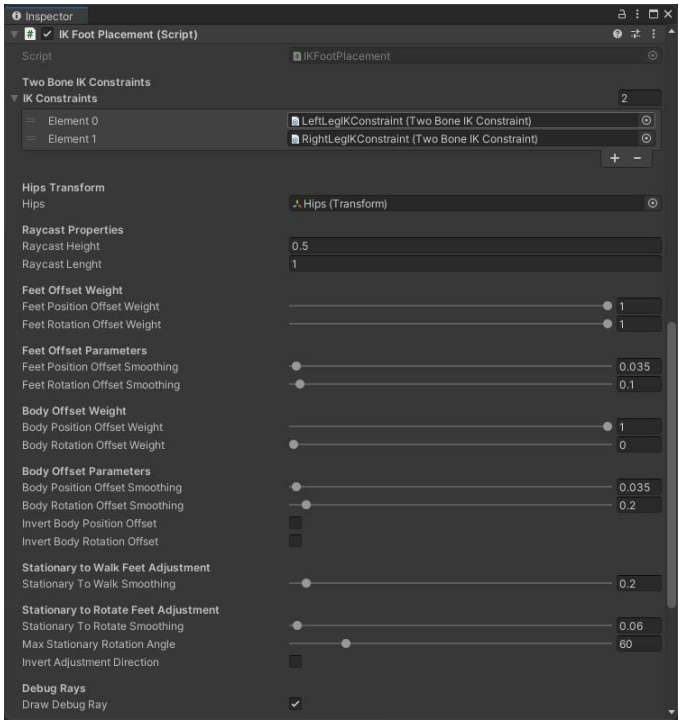
1. Overview

IKFootPlacement is a C# script for Unity engine that controls the position and rotation of character's feet, relative to the ground beneath. This script works by using inverse kinematics (IK) provided with the Animation Rigging package from Unity.



Pic. 1. Screenshot from a demo scene

The script supports characters with any number of legs, which means it does not require the usage of a humanoid avatar. Instead it works with the use of Two Bone IK Constraint provided with the the Animation Rigging package.



Pic. 2. Example configuration of the IKFootPlacement script on a bipedal character

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

2. Package Contents

This package contains:

- C# code:
  - IKFootPlacement.cs script that attaches to the GameObject and provides the core functionality of this package,
  - IKSetup.cs script that automatically configures the Two Bone IK Constraints and attaches the IKFootPlacement script to the GameObject,
  - IKFootPlacementJob.cs struct that calculates and applies the position and rotation offset for each foot,
  - BasicCharacterMovement.cs and BasicCharacterRotation.cs scripts that controls the movement of a demo character, - Other, additional code.
- Demo 3D models and animations:
  - Alpha is a simple, bipedal character that is a part of Unity's „Starter Assets: Character Controllers" package (assetstore.unity.com),
  - Simple character locomotion animations created by Lifelike Motion.
- Demo prefabs and scenes:
  - 4 example prefabs that showcase the IKFootPlacement script working on a bipedal characters,
  - DemoScene to test out the IKFootPlacement script on a variety of surfaces (uneven terrain, ramp, rotating platforms, etc.).

Please note that this asset uses code from IKFootPlacement repository (github.com/plonkabartosz/IKFootPlacement.git), created by Bartosz Plonka, licensed under MIT Open Source license (https://opensource.org/license/mit). See Third Party Notices.txt file in the package for more details.

3. Package Requirements

3.1. Package requirements and dependencies

These are the following requirements and dependencies for the **IKFootPlacement - Free IK Script** package:

- Unity engine:

  - Version 2021.2.4 or higher is recommended (Unity 2021.2.4 Website),

  - Version 2020.2.0 is a minimum (Unity 2020.2.0 Website).
- Animation Rigging package:

  - Version 1.1.1 or higher is recommended (Animation Rigging 1.1 Manual),

  - Version 1.0.3 is a minimum (Animation Rigging 1.0 Manual).

3.2. Optional packages

You may also download and import these optional packages:

- FBX Exporter package:

  - Any version that supports Unity 2020.2.0 or higher is recommended (FBX Exporter 4.2.1 Manual)

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

4. Installation and Configuration

4.1. Package import

To correctly import the IKFootPlacement package, make sure to follow these steps:

- Download and import the package from the package manager:
  - Go to **Window > Package Manager**,
  - Choose **Packages: My Assets**,
  - Choose **IKFootPlacement - Free IK Script** package,
  - Click **Download**,
  - After downloading the package click **Import**,
  - After that a pop-up will appear, asking if You want to install or upgrade the package dependencies (see pic. 3.). If you have the **Animation Rigging** package already installed, then You can skip this procedure. If not, then click on **Install/Upgrade** button to automatically add needed dependencies to Your project,
  - Choose the content You would like to import.
- Make sure that You have imported the **Tool** folder, as it contains all of the needed code.

4.2. Script configuration

To correctly configure the IKFootPlacement script, make sure to follow these steps:

- Make sure that You have imported the **Tool** folder, as it contains all of the needed code,
- Make sure that Your skinned character legs joints follow the correct naming scheme:

  For bipedal characters:

```
Hips
  LeftUpLeg
          LeftLeg
                  LeftFoot
  RightUpLeg
          RightLeg
                  RightFoot
```


Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

For other character (e.g. quadrupedal):

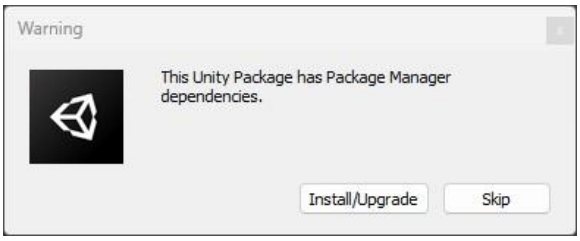
```
Hips
        LeftUpLeg1
                LeftLeg1
                        LeftFoot1
        LeftUpLeg2
                LeftLeg2
                LeftFoot2
        (…)
        RightUpLeg1
                RightLeg1
                        RightFoot1
        (…)
```

- Make sure that the project has the **Animation Rigging** package installed,
- Create a new, empty scene,
- Import Your skinned character into that scene (see pic. 4.↗),
- Attach an **Animator** component to the character (see pic. 4.↗),
- Attach the **IKSetup** script to the character (see pic. 5.↗) in one of two ways:
  - drag and drop the script from the **Tool** folder,
  - click on the character in the hierarchy tab, then in the inspector tab click **Add Component** and search for **IKSetup**.
- Follow the steps that are shown in the attached script (see pic. 5.↗),
- Click on **Find References** button to automatically search for the character's joints (see pic. 6.↗),
- If You are having troubles automatically finding the joints with the **Find References** button (e.g. joints use different naming scheme), You can always attach the references manually,
- When all of the references are attached, You can click on **Start Setup** button, to begin the automatic setup procedure,
- Examine the following:
  - Check the console logs. There should be no errors or warnings,
  - Check the hierarchy tab. The character should now have a new set of objects,
  - Check the inspector tab. The character should now have a new set of components attached (RigBuilder, Rig, IKFootPlacement)
  - In the inspector tab, see the **IKFootPlacement** script. Check if the **IK Constraint** list is populated correctly (every constraint should represent each leg of the character),

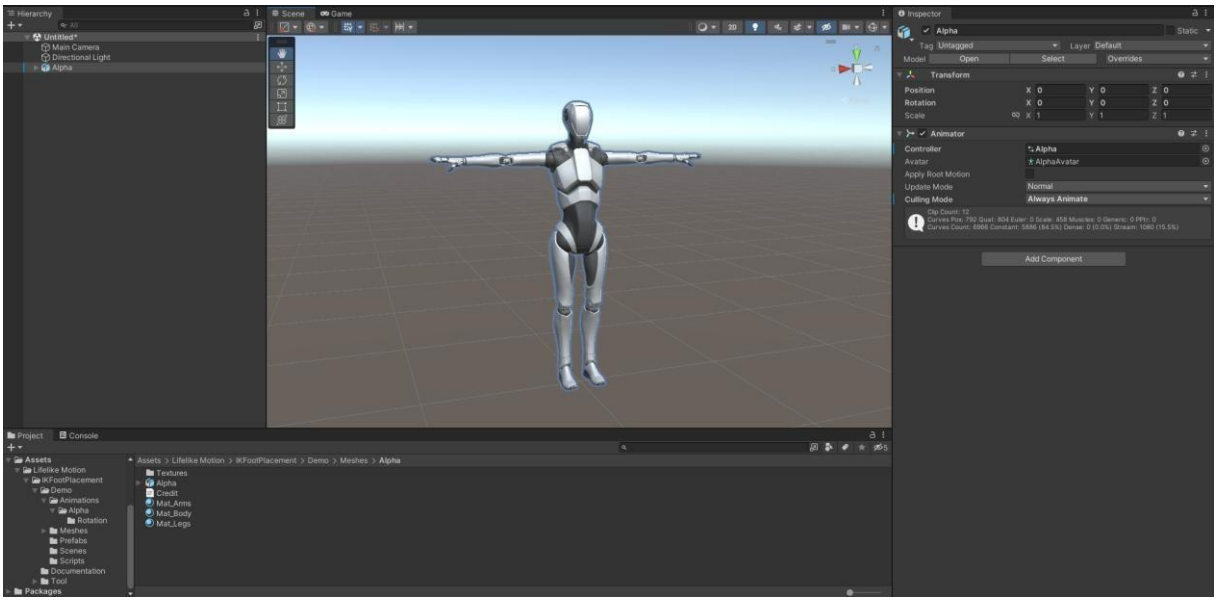
If all of the steps are done properly, You should have a correctly configured IKFootPlacement script (see pic. 7.↗).

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

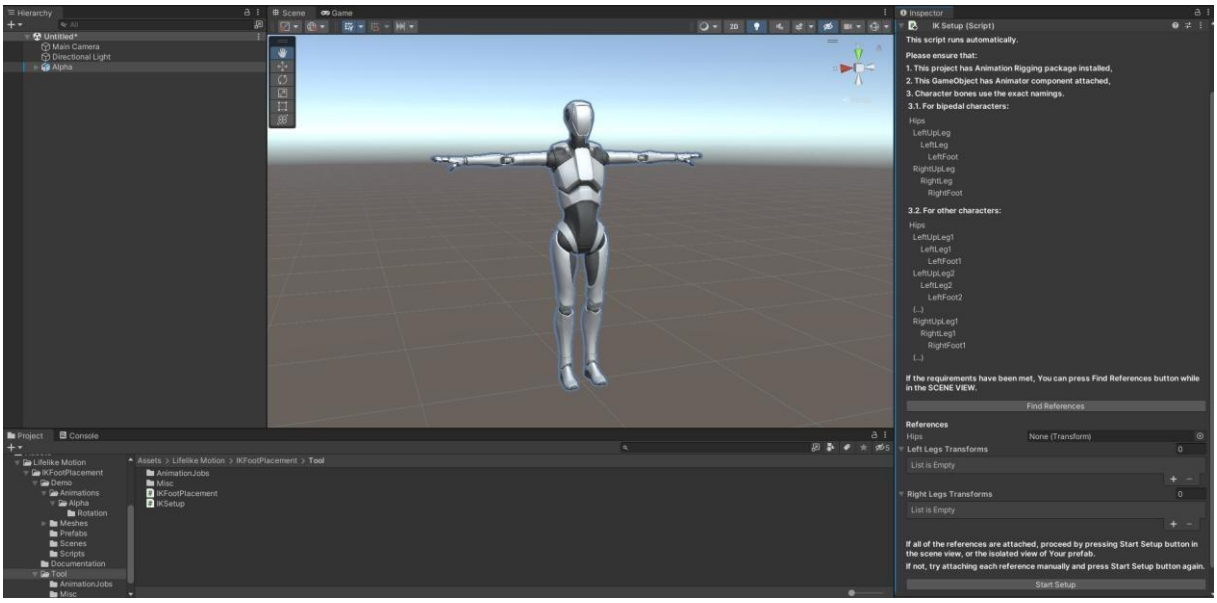
If You had any troubles with the configuration procedure, please refer to the **Troubleshooting** (chapter 6.↗) part of this documentation.



Pic. 3. Installing needed dependencies

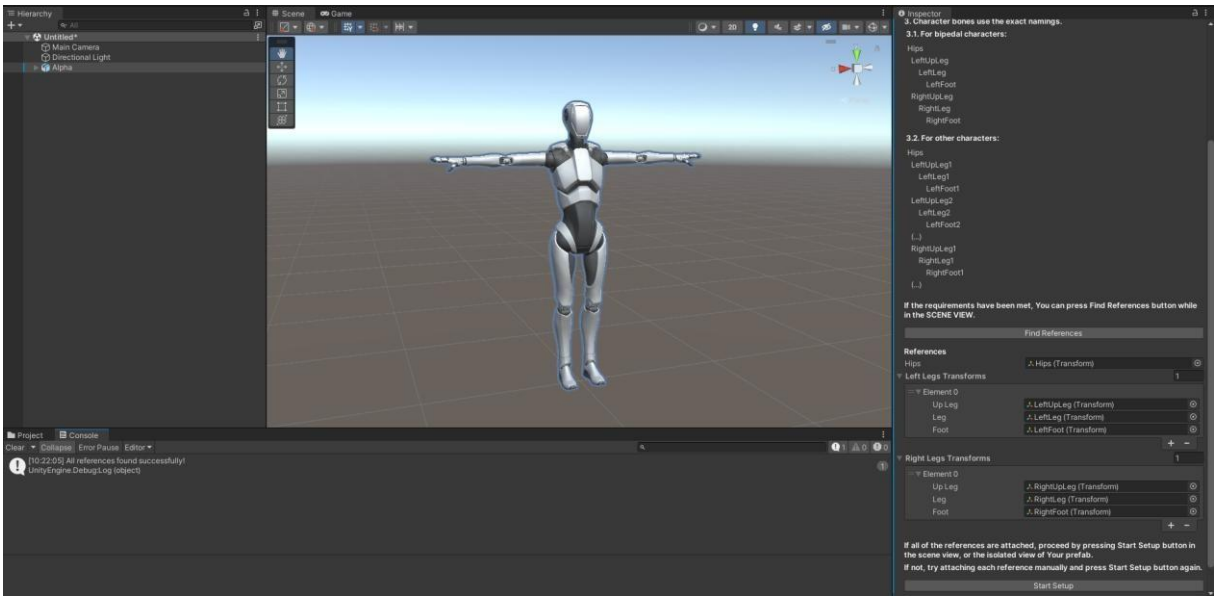


Pic. 4. Importing a demo character into an empty scene

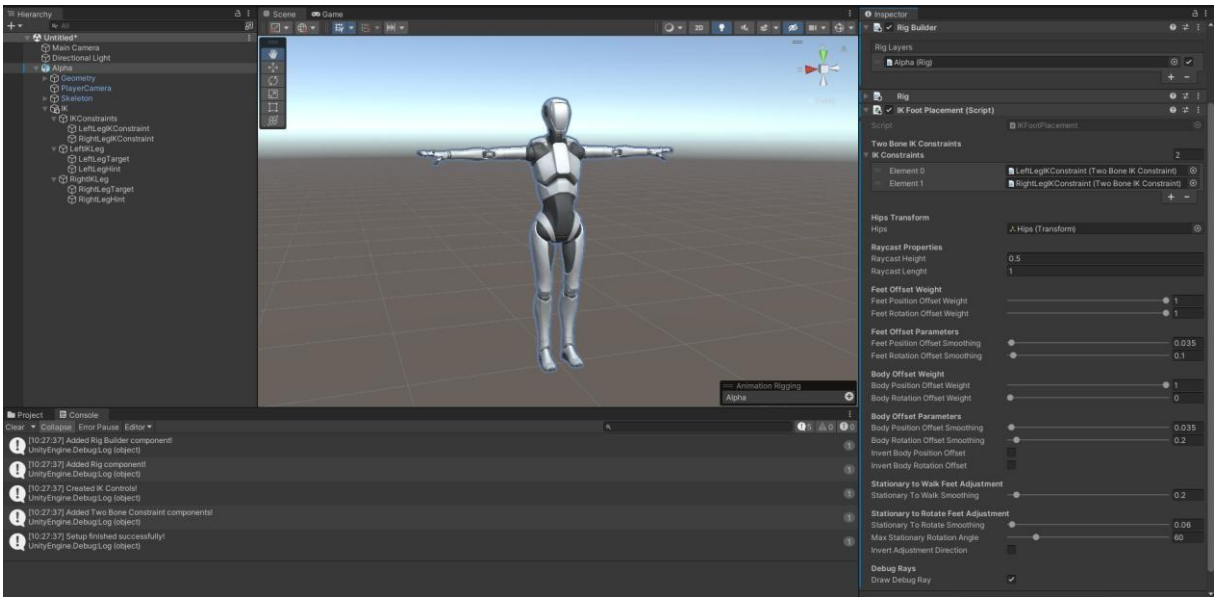


Pic. 5. Attaching the IKSetup script to the demo character

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)



Pic. 6. The IKSetup script automatically finds all of the needed references



Pic. 7. Automatic setup procedure finished successfully

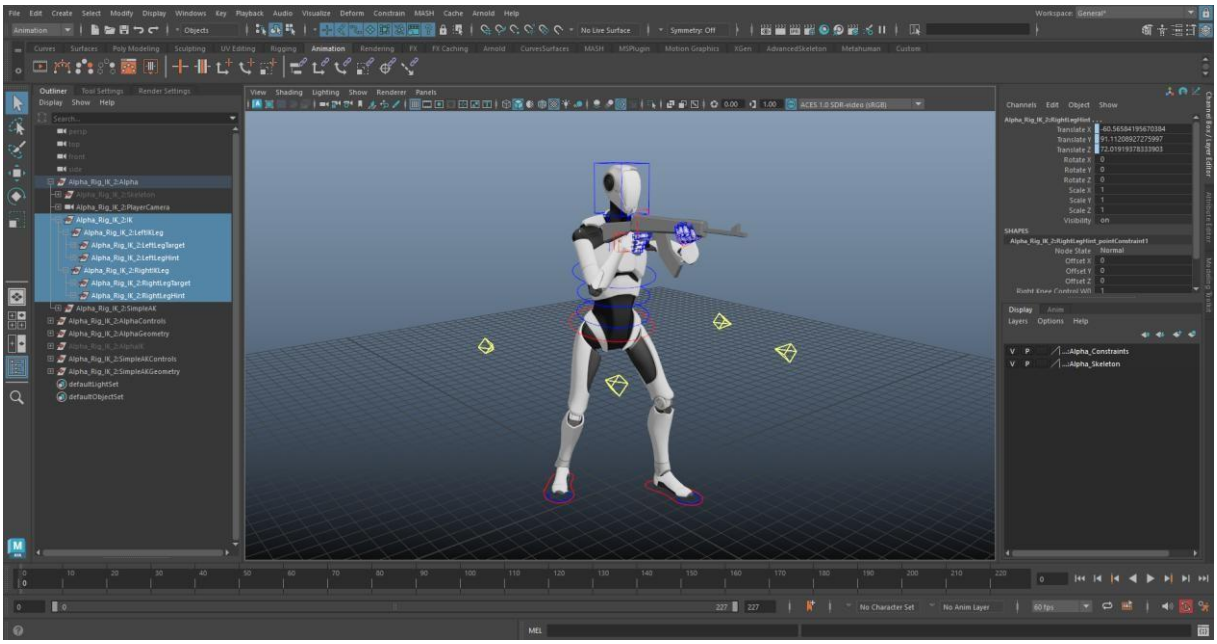
Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

5. Usage

5.1. Baking the animation onto the IK controls

It is important to note, that the IKFootPlacement script **does not** animate the IK controls (targets and hints). In order to use the IKFootPlacement script correctly, You need to animate the IK controls and store this animation data inside of animation clips.

In order to do that, You may need to use another software like Blender, Autodesk Maya, etc. and bake the animation onto these IK controls (see pic. 8.). Use the **FBX Exporter** package features to export the IK controls and import them to Your software of choice.



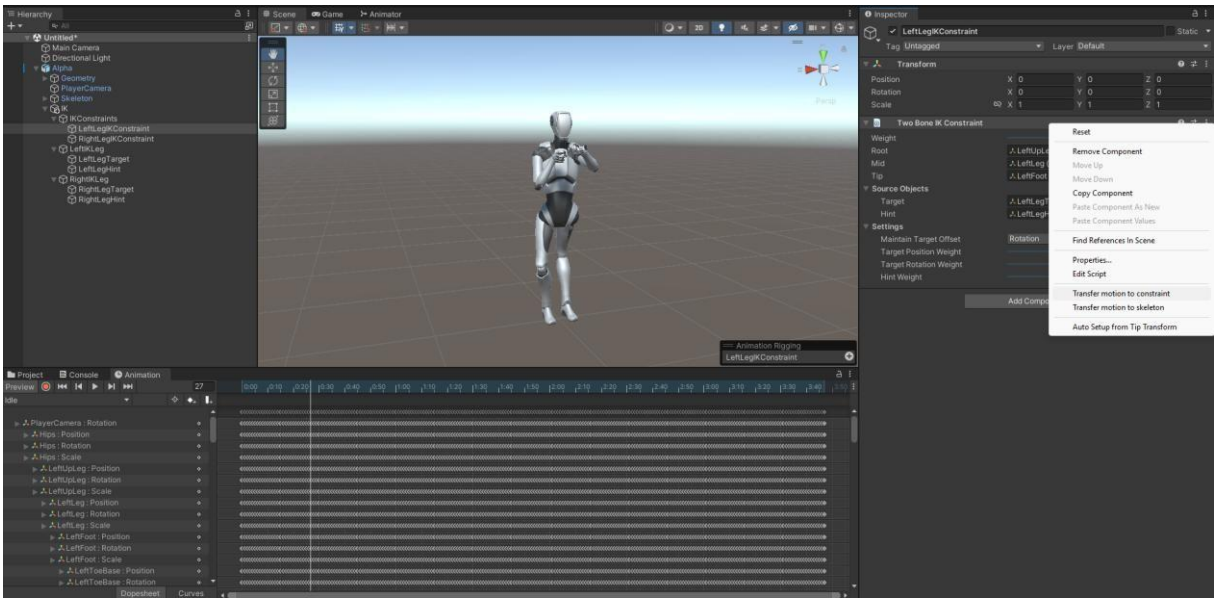
Pic. 8. Baking the animation onto IK controls with Autodesk Maya software

It is possible (but not recommended due to bad retargeting quality) to use Animation Rigging package features to retarget the joint movement onto the IK controls inside the Unity engine:

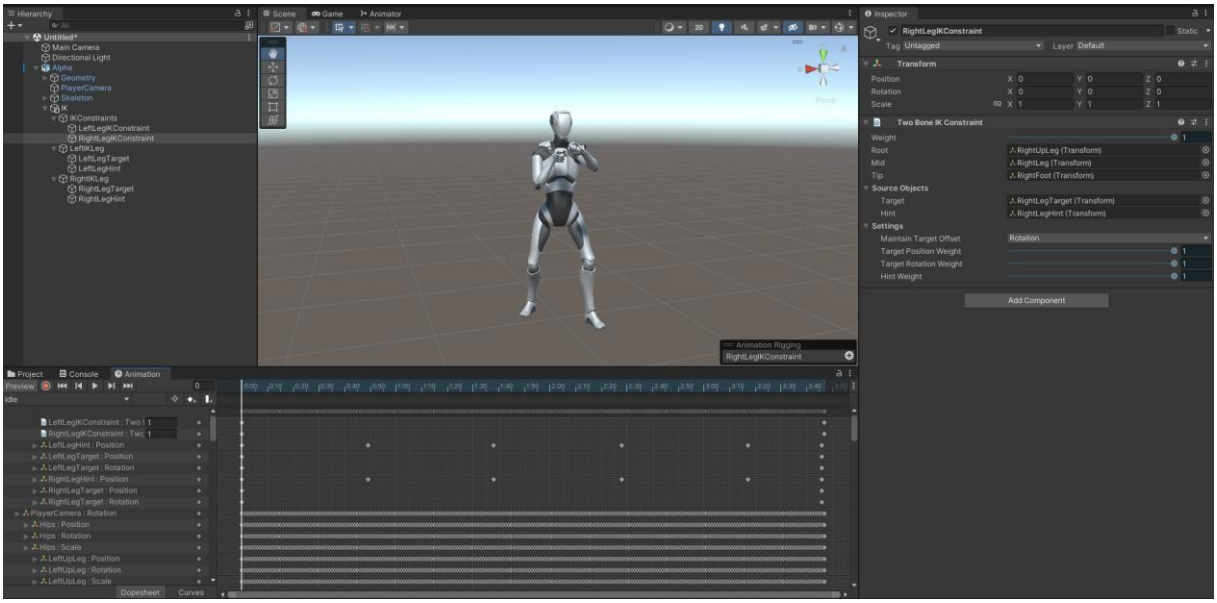
- Please make sure that Your existing character animations are not read-only files (e.g. extract the animation clips from the FBX file),
- Open the Animation tab by going to **Window > Animation > Animation**,
- In the character's hierarchy look for **IKConstraints** object,
- Inside of it You will find number of constraints that are associated with each leg of the character,
- Select first **IKConstraint** object,

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

- In the **Animation tab** click on **Preview** button to enable the preview of the selected animation,
- In the **Inspector** tab, right click on the **Two Bone IK Constraint** component,
- Click on **Transfer motion to constraint** option (see pic. 9.↗),
- The animation data should now be baked onto the IK controls and saved in the selected animation clip (see pic. 10.↗),
- Continue this process through every animation for this character.
- For more information check the official documentation of Bidirectional Motion Transfer (https://docs.unity3d.com/Packages/com.unity.animation.rigging@1.1/manual/BidirectionalMotionTransfer.html↗).



Pic. 9. Transferring motion to the IK constraint



Pic. 10. Motion baked onto the IK controls

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

5.2. Using the IKFootPlacement script

IKFootPlacement script has **three public boolean variables** that controls the movement logic of the script:

- **isGrounded** is a boolean that represents the grounded status of character collider,
- **isMoving** is a boolean that is true when the character is moving in the scene,
- **jumped** is a boolean that is true when a character is in a jump state.

These variables have to be controlled driectly by another script that controls the character movement. I have created two basic scripts that controls the character movement and rotation:

- BasicCharacterMovement.cs,
- BasicCharacterRotation.cs.

These scripts implement the basic functionality and show how these variables can be controled:

```
if (iKFootPlacement != null)
{
      iKFootPlacement.isGrounded = false;
      iKFootPlacement.jumped = true;
}
```

Without that control the script **will not** work properly.

List of IKFootPlacement script public variables:

- **Two Bone IK Constraints:**
  - iKConstraints – List of Two Bone IK Constraints that represent each leg, that is controlled by inverse kinematics.
- **Hip Transform:**
  - hips - Element of character's skeleton that represents the hips.
- **Raycast properties:**
  - raycastHeight - float value that controls the height from which raycasts are casted downwards,
  - raycastLength - float value that controls the length (depth) of casted raycasts.
- **Feet offset weight:**
  - feetPositionOffsetWeight - float value that controls the weight of the position offset applied to the IK controls transforms,
  - feetRotationOffsetWeight - float value that controls the weight of the rotation offset applied to the IK controls transforms.
- **Feet offset parameters:**

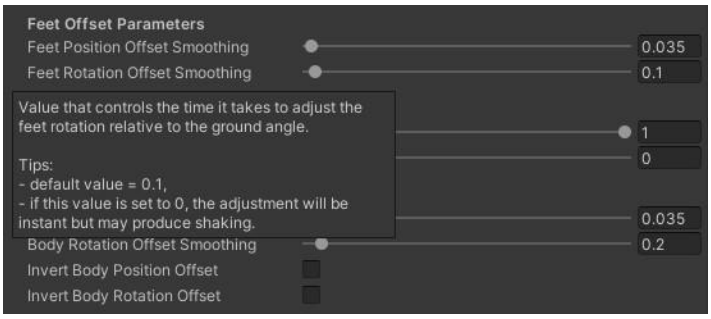
Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

- feetPositionOffsetSmoothing - float value that controls the time it takes to adjust the feet position relative to the ground height,
- feetRotationOffsetSmoothing - float value that controls the time it takes to adjust the feet rotation relative to the ground angle.

- **Body offset weight:**
  - bodyOffsetPositionWeight - float value that controls the weight of the position offset applied to the hips transform,
  - bodyOffsetPositionWeight - float value that controls the weight of the rotation offset applied to the hips transform.
- **Body offset parameters:**
  - bodyPositionOffsetSmoothing - float value that controls the time it takes to offset the "Hips" transform position relative to the lowest position of the feet,
  - bodyRotationOffsetSmoothing - float value that controls the time it takes to offset the "Hips" transform rotation relative to the rotation of the ground beneath that transform,
  - invertBodyPositionOffset - boolean that controls the direction in which the hips transform position is offset,
  - invertBodyRotationOffset - boolean that controls the direction in which the hips transform rotation is offset.
- **Stationary to walk feet adjustment:**
  - stationaryToWalkSmoothing - float value that controls the time it takes to adjust the IK Controls from a stationary state to a walking state.
- **Stationary to rotate feet adjustment:**
  - stationaryToRotateSmoothing - float value that controls the time it takes to adjust the feet position and rotation when in a stationary state but rotated above "MaxStationaryRotationAngle" value,
  - maxStationaryRotationAngle - float value that controls the cone angle in which the IK Controls remain in a stationary state,
  - invertAdjustmentDirection - boolean that controls the direction of feet adjustment, when rotating the character but not moving.
- **Debug rays:**
  - drawDebugRay - boolean that controls the rendering of the debug rays visible only in the editor.

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

Every single of these parameters above has a description that will show up when You hover over a chosen parameter with Your mouse cursor (see pic. 11.).



Pic. 11. Description of the Feet Rotation Offset Smoothing parameter

- **Control booleans (Hidden in the inspector)**
  - isActive - boolean that controls wether the script should cast rays, calculate and apply the offset,
  - startup - boolean that if set to true will make the IKFootPlacement script go through the startup procedure as if it was the Start() method,
  - jumped - boolean that is true when a character is in a jump state,
  - isMoving - boolean that is true when the character is moving in the scene,
  - isGrounded - boolean that represents the grounded status of character collider,

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)

6. Troubleshooting

These are the potential troubles You may encounter using the IKFootPlacement package:

- *„I can't change the naming scheme of my joints, but I still want to use the automatic reference finding feature."*

  No problem, all You need to do is open up the **IKSetup.cs** with a code editor of Your choice. Then search for these lines of code:

  ```
  public string hipsName = "Hips";


  public string leftUpLegName = "LeftUpLeg";
  public string leftLegName = "LeftLeg";
  public string leftFootName = "LeftFoot";


  public string rightUpLegName = "RightUpLeg";
  public string rightLegName = "RightLeg";
  public string rightFootName = "RightFoot";
  ```

  Here You can easily modify the naming scheme to Your personal preferences.

If You are having other troubles with importing, configuring or running the IKFootPlacement package contents, please leave a comment on this package Unity Asset Store page or contact me directly via the official website:

Lifelike Motion - https://www.lifelikemotion.com.pl/support

You can also find me on my social media:

LinkedIn - https://www.linkedin.com/company/lifelike-motion

Instagram - https://www.instagram.com/lifelike_motion

Bluesky - https://bsky.app/profile/lifelikemotion.bsky.social

Facebook - https://www.facebook.com/lifelikemotionmocap

Original Author: Bartosz Plonka
Source: https://github.com/plonkabartosz/IKFootPlacement.git
License: MIT Open Source (https://opensource.org/license/mit)